

Mind Space — Technical Requirements Document

Document 02 of 06 | Version 1.0 | PFE Project

Tech stack overview

Layer	Technology	Rationale
Frontend	Flutter (Dart) — Android target	Cross-platform foundation, native performance, Riverpod for state management
Backend runtime	Supabase Edge Functions (Deno / TypeScript)	Serverless, co-located with DB, zero separate backend to deploy
Database	PostgreSQL via Supabase (pgvector extension enabled)	Vector similarity search native, RLS built-in, real-time subscriptions available
Auth	Supabase Auth — Google OAuth + email magic link	No passwords. JWT tokens. Auto profile creation via Postgres trigger.
LLM — chat + reflection	Groq API: llama-3.3-70b-versatile	Free tier (1000+ req/day), OpenAI-compatible, fastest inference available
LLM — safety classifier + arc naming	Groq API: llama-3.1-8b-instant	Free tier (14,400 req/day), fast enough for pre-message classification
LLM — arc insight synthesis	OpenRouter: deepseek/deepseek-r1:free	Stronger reasoning for long-context synthesis; 200 req/day sufficient for insights
Embeddings	OpenAI text-embedding-3-small	\$0.02/1M tokens (~\$0.01 total for PFE), 1536 dimensions, reliable cold start
Vector search	pgvector in Supabase (<=> cosine distance operator)	Native to Postgres, no separate vector DB needed
File storage	Supabase Storage	Spirit assets bundled in app; storage only needed for potential future media
Hosting	Supabase (DB + Edge Functions), APK distributed manually	Zero infrastructure cost for PFE

Flutter project structure

```
lib/  
  main.dart           # App entry, theme, GoRouter setup  
  core/  
    theme/  
    spacing           # Design tokens: colors, typography,
```

components/ EmotionSpirit, Card	# Shared widgets: MindSpaceButton,
constants/	# App-wide constants, spirit definitions
features/	
auth/	# Auth screens + Riverpod providers
home/	# Home screen + ambient orb
chat/	# Chat modal + streaming SSE consumer
reflection/	# Post-chat + read-only reflection screens
history/	# Timeline + Arcs grid screens
arc_detail/	# Arc detail + insight screen
settings/	# Settings screen
services/	
supabase_service.dart	# Supabase client singleton
llm_service.dart	# Edge function callers
arc_service.dart	# Arc assignment + centroid logic
models/	# Dart data classes for all DB entities

Key Flutter dependencies

Package	Version	Purpose
supabase_flutter	^2.x	Supabase client, auth, realtime, storage
flutter_riverpod	^2.x	State management — providers for auth, chat, arcs
go_router	^13.x	Declarative routing, deep links, modal routes
flutter_animate	^4.x	Declarative animations — reflection cascade, orb pulse
flutter_dotenv	^5.x	Environment variable loading from .env
http	^1.x	SSE stream consumption for Groq streaming
shared_preferences	^2.x	Onboarding seen flag, local user preferences
flutter_haptic_feedback	^0.x	Haptic on send, on reflection arrival

Edge Functions list

Function name	Trigger	Steps	LLM used
chat-stream	POST from Flutter during chat	1. Save user message 2. Fetch history 3. Stream Sage reply via SSE 4. Save assistant message	llama-3.3-70b-versatile (Groq)
safety-check	Called before chat-stream processes	1. Classify message SAFE/CRISIS 2. Return flag	llama-3.1-8b-instant (Groq)

Function name	Trigger	Steps	LLM used
end-chat	POST on Wrap Up tap	1. Generate reflection JSON 2. Embed reflection text 3. Call assign-arc 4. Return reflection_id + arc_id	llama-3.3-70b-versatile + OpenAI embed
assign-arc	Called by end-chat	1. Fetch user arcs 2. Cosine similarity vs centroids 3. Assign or create arc 4. Update centroid 5. Name if new (8B)	llama-3.1-8b-instant (naming only)
generate-arc-insight	POST from Arc Detail screen	1. Fetch arc sessions in order 2. Build cross-arc context 3. Synthesize insight 4. Cache result	deepseek/deepseek-r1:free (OpenRouter)

Environment variables

Variable	Where stored	Used by
SUPABASE_URL	Flutter .env (public)	supabase_flutter init
SUPABASE_ANON_KEY	Flutter .env (public)	supabase_flutter init
SUPABASE_SERVICE_ROLE_KEY	Supabase Edge Function secrets	Edge Functions only — never in Flutter
GROQ_API_KEY	Supabase Edge Function secrets	chat-stream, safety-check, end-chat, assign-arc
OPENROUTER_API_KEY	Supabase Edge Function secrets	generate-arc-insight
OPENAI_API_KEY	Supabase Edge Function secrets	end-chat (embeddings)

LLM API configuration

Groq (chat + reflection)

```

Base URL: https://api.groq.com/openai/v1
Model (chat/reflection): llama-3.3-70b-versatile
Model (safety/naming): llama-3.1-8b-instant
Temperature (chat): 0.7
Temperature (reflection): 0.5
Temperature (safety): 0.1
Temperature (naming): 0.3
Max tokens: 1000

```

OpenRouter (arc insight)

```
Base URL: https://openrouter.api.ai/api/v1
Model:    deepseek/deepseek-r1:free
Temperature: 0.3 (deterministic synthesis)
Max tokens: 1500
```

OpenAI (embeddings)

```
Model:    text-embedding-3-small
Dimensions: 1536
Max input: 8191 tokens (embed what_sage_heard only, not full chat)
```

Hard technical constraints

- All LLM keys must live in Edge Function secrets — never in Flutter source or .env
- RLS (Row Level Security) must be enabled on all tables before any data is written
- pgvector extension must be enabled on Supabase project before embeddings are stored
- Groq free tier: 1000 RPD for 70B model — do not use for safety check (use 8B)
- OpenRouter free tier: 200 RPD — use only for arc insight generation
- Cosine similarity threshold: 0.78 for arc assignment (tune empirically; see PRD eval criteria)
- Embedding vector dimensions must match model output — do not mix embedding models
- The safety classifier must fail closed: if the API is down, block the chat and show resources
- APK target: Android API 24+ (Android 7.0). Test on physical device, not emulator only